Practical No: 1

Aim: Implementing Substitution and Transposition Ciphers:

Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

1.1 Substitution Cipher :

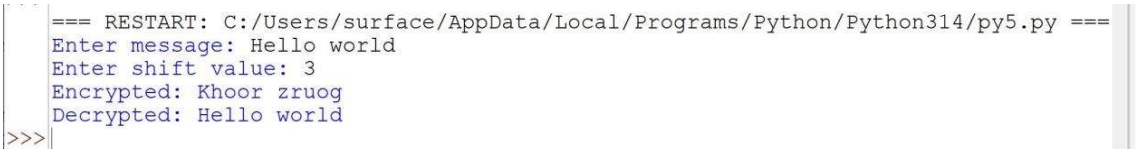
1.1.1 (Caesar Cipher)

Code:

```
def encrypt Caesar(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
            else:
                result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += char
    return result:
def decrypt_caesar(text, shift):
    return encrypt_caesar(text, -shift)
message = input("Enter message: ")
shift = int(input("Enter shift value: "))
```

```
encrypted = encrypt_caesar(message, shift)
print("Encrypted:", encrypted)
decrypted = decrypt_caesar(encrypted, shift)
print("Decrypted:", decrypted)
```

Output:



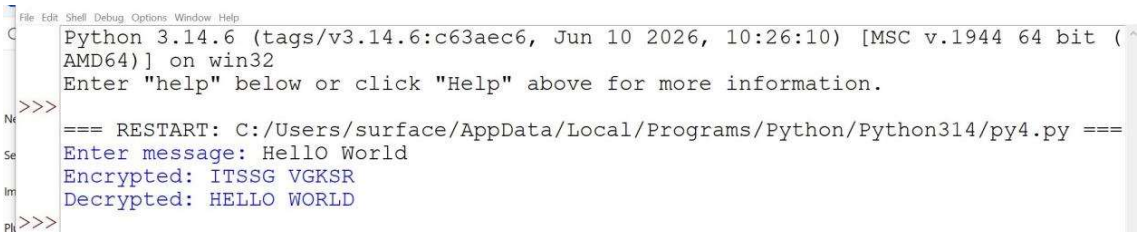
1.1.2 (Monoalphabetic Cipher)

Code:

```
plain_alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher_alphabet = "QWERTYUIOPASDFGHJKLZXCVBNM"
def encrypt(text):
    result = ""
    for char in text.upper():
        if char in plain_alphabet:
            index = plain_alphabet.index(char)
            result += cipher_alphabet[index]
        else:
            result += char
    return result
def decrypt(text):
    result = ""
```

```
    for char in text.upper():
        if char in cipher_alphabet:
            index = cipher_alphabet.index(char)
            result += plain_alphabet[index]
        else:
            result += char
    return result
message = input("Enter message: ")
encrypted = encrypt(message)
print("Encrypted:", encrypted)
decrypted = decrypt(encrypted)
print("Decrypted:", decrypted)
```

Output:



11.2 (Playfair Cipher)

Code:

```
def generate_matrix(key):
key = key.upper().replace("J", "I")
matrix = []
used = set()
```

```
for char in key:
if char.isalpha() and char not in used:
matrix.append(char)
used.add(char)

for char in "ABCDEFGHIKLMNOPQRSTUVWXYZ": # J omitted
if char not in used:
matrix.append(char)
used.add(char)
return [matrix[i:i+5] for i in range(0, 25, 5)]
def find_position(matrix, char):
for row in range(5):
for col in range(5):
if matrix[row][col] == char:
return row, col
def prepare_text(text):
text = text.upper().replace("J", "I")
text = ''.join(filter(str.isalpha, text))
result = ""
i = 0
while i < len(text):
a = text[i]
b = text[i + 1] if i + 1 < len(text) else 'X'
```

```
if a == b:
result += a + 'X'
i += 1
else:
result += a + b
i += 2
if len(result) % 2 != 0:
result += 'X'
return result
def encrypt(text, matrix):
text = prepare_text(text)
cipher = ""
for i in range(0, len(text), 2):
a, b = text[i], text[i+1]
r1, c1 = find_position(matrix, a)
r2, c2 = find_position(matrix, b)
if r1 == r2: # Same row
cipher += matrix[r1][(c1 + 1) % 5]
cipher += matrix[r2][(c2 + 1) % 5]
elif c1 == c2: # Same column
cipher += matrix[(r1 + 1) % 5][c1]
cipher += matrix[(r2 + 1) % 5][c2]
else: # Rectangle
```

```
cipher += matrix[r1][c2]
cipher += matrix[r2][c1]
return cipher
def decrypt(ciphertext, matrix):
plaintext = ""
for i in range(0, len(ciphertext), 2):
a, b = ciphertext[i], ciphertext[i + 1]
r1, c1 = find_position(matrix, a)
r2, c2 = find_position(matrix, b)
if r1 == r2: # Same row
plaintext += matrix[r1][(c1 - 1) % 5]
plaintext += matrix[r2][(c2 - 1) % 5]
elif c1 == c2: # Same column
plaintext += matrix[(r1 - 1) % 5][c1]
plaintext += matrix[(r2 - 1) % 5][c2]
else: # Rectangle
plaintext += matrix[r1][c2]
plaintext += matrix[r2][c1]
return plaintext
key = input("Enter keyword: ")
plaintext = input("Enter plaintext: ")
matrix = generate_matrix(key)
print("\nPlayfair Matrix:")
```

```
for row in matrix:

print(" ".join(row))

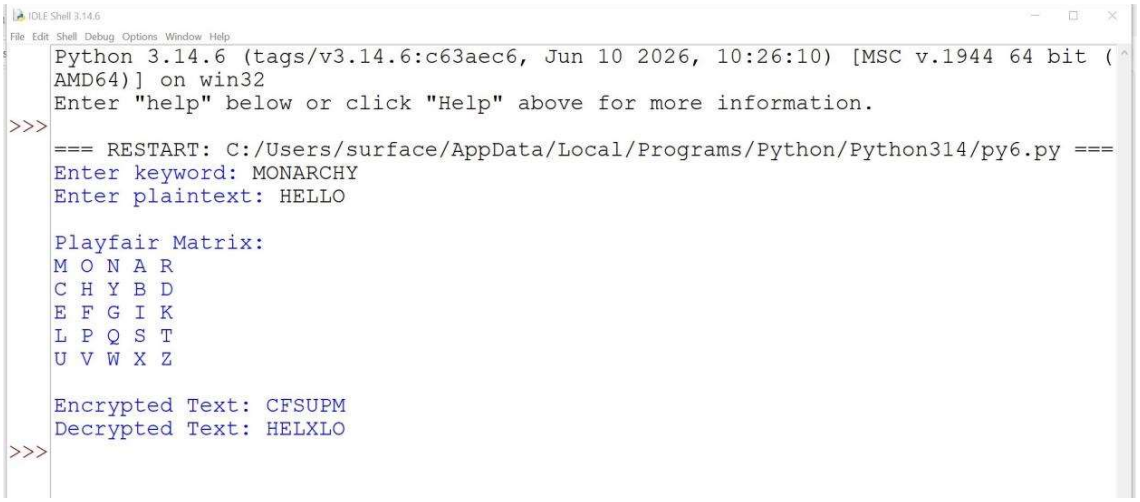
ciphertext = encrypt(plaintext, matrix)

print("\nEncrypted Text:", ciphertext)

decrypted = decrypt(ciphertext, matrix)

print("Decrypted Text:", decrypted)
```

Output:



1.2 Transposition Techniques

1.2.1 (Rail fence Cipher)

Code:

```
def encrypt(text, rails):

 fence = [" for _ in range(rails)] row = 0 direction = 1

for char in text:
    fence[row] += char

    if row == 0:
        direction = 1
    elif row == rails - 1:
```

```
        direction = -1
    row += direction
return ''.join(fence)


def decrypt(cipher, rails):

# Create zigzag pattern positions

 pattern = [['' for _ in range(len(cipher))]

for _ in range(rails)]

row = 0
direction = 1
for col in range(len(cipher)):
    pattern[row][col] = '*'
    if row == 0:
        direction = 1
    elif row == rails - 1:
        direction = -1
    row += direction

# Fill ciphertext into pattern
index = 0
for i in range(rails)
    for j in range(len(cipher)):
        if pattern[i][j] == '*' and index < len(cipher):
            pattern[i][j] = cipher[index]
            index += 1

# Read zigzag pattern
result = ''
row = 0
direction = 1
for col in range(len(cipher)):
    result += pattern[row][col]
    if row == 0:
        direction = 1
```

```
    elif row == rails - 1:
        direction = -1
    row += direction

text = input("Enter plaintext: ")

rails = int(input("Enter rails: "))

cipher = encrypt(text, rails)

print("Encrypted:", cipher)

plain = decrypt(cipher, rails)

print("Decrypted:", plain)
```

Output:

```
>>>
    === RESTART: C:/Users/surface/AppData/Local/Programs/Python/Python314/PY7.py ===
    Enter plaintext:
    === RESTART: C:/Users/surface/AppData/Local/Programs/Python/Python314/PY7.py ===
    Enter plaintext: HELLO WORLD
    Enter rails: 3
    Encrypted: HOREL OLLWD
    Decrypted: HELLO WORLD
>>>
```

1.2.2 (Columnar Transposition Cipher)

Code :

```
import math

def get_key_order(keyword):

 keyword = keyword.upper()

 pairs = sorted([(char, i) for i, char in enumerate(keyword)])

 order = [0] * len(keyword)

for rank, (_, index) in enumerate(pairs, start=1):
    order[index] = rank
order return

def encrypt(text, keyword):
```

```
key = get_key_order(keyword)

 text = text.upper().replace(" ", "_")

cols = len(key)
rows = math.ceil(len(text) / cols)

text += "_" * (rows * cols - len(text))

matrix = [list(text[i:i + cols])

 for i in range(0, len(text), cols)] cipher = ""
for num in range(1, cols + 1):
    col = key.index(num)
    for row in matrix:
        cipher += row[col]
return cipher

def decrypt(cipher, keyword): key = get_key_order(keyword)

cols = len(key)
rows = len(cipher)
matrix = [[" for _ in range(cols)] for _ in range(rows)]]
k = 0
for num in range(1, cols + 1):
    col = key.index(num)
    for row in range(rows):
        matrix[row][col] = cipher[k]
        k += 1
plaintext = ""
for row in matrix

  plaintext += ".join(row)
return plaintext.rstrip('_')

keyword = input("Enter Keyword: ")

plaintext = input("Enter Plaintext: ")

print("Numeric Key:", get_key_order(keyword))
```

```
cipher = encrypt(plaintext, keyword)

 print("Encrypted Text:", cipher)

decrypted = decrypt(cipher, keyword)

 print("Decrypted Text:", decrypted)
```

Output:

```
======= RESTART: C:/Users/surface/AppData/Local/Programs/Python/Python314/PY`10.py ======
Enter Keyword: HACK
Enter Plaintext: HELLO WORLD
Numeric Key: [3, 1, 2, 4]
Encrypted Text: E_LLWDHORLO_
Decrypted Text: HELLO_WORLD
>>>
```

1.2.2 (Double Columnar Transposition Cipher)

Code:

```
import math

def get_key_order(keyword):

 keyword = keyword.upper()

 pairs = sorted([(char, i)

 for i, char in enumerate(keyword)]) order = [0] * len(keyword)

for rank, (_, index) in enumerate(pairs, start=1):
    order[index] = rank  order

def encrypt(text, keyword): key = get_key_order(keyword) text =
text.upper().replace(" ", "_")

cols = len(key)
rows = math.ceil(len(text) / cols)
return
text += "_" * (rows * cols - len(text))
matrix = [list(text[i:i + cols]) for i in range(0, len(text), cols)]
r cipher = ""
for num in range(1, cols + 1):
```

```
    col = key.index(num)
    for row in matrix:
        cipher += row[col]
return cipher

def decrypt(cipher, keyword): key = get_key_order(keyword)

cols = len(key)
rows = len(cipher) // cols
matrix = [[" for _ in range(cols)] for _ in range(rows)]
k = 0
for num in range(1, cols + 1):
    col = key.index(num)
    for row in range(rows):
        matrix[row][col] = cipher[k]
        k += 1
plaintext = ""
for row in matrix:
    plaintext += ''.join(row)
return plaintext.rstrip('_')

keyword = input("Enter Keyword: ") plaintext = input("Enter Plaintext: ")

cipher1 = encrypt(plaintext, keyword) cipher2 = encrypt(cipher1, keyword)

print("\nFirst Encryption :", cipher1)

print("Double Encryption:", cipher2)

plain1 = decrypt(cipher2, keyword) plain2 = decrypt(plain1, keyword)

print("\nAfter 1st Decryption:", plain1)

print("Original Plaintext :", plain2)
```

Output:

```
>>>
======== RESTART: C:/Users/surface/AppData/Local/Programs/Python/Python314/PY8.py =======
Enter Keyword: HACK
Enter Plaintext: HELLO WORLD

First Encryption : E_LLWDHORLO_
Double Encryption: _DLLHOEWRLO_

After 1st Decryption: E_LLWDHORLO
Original Plaintext : WELHD_LO
>>>
```